DroPE
Отчет о частичном воспроизведении

Статья: Extending the Context of Pretrained LLMs by Dropping Their Positional Embeddings
Yoav Gelberg, Koshi Eguchi, Takuya Akiba, Edoardo Cetin
arXiv:2512.12167v1, 2025. Локальный запуск: 9 октября 2026 года.

1. Цель и основной результат

Цель работы: запустить опубликованный код авторов локально, проверить механизм удаления позиционных эмбеддингов и выполнить сокращенный эксперимент на pretrained SmolLM-360M. Дополнительно проверен доступ к выпущенному авторами checkpoint Llama-2-7B-DroPE. Полные LongBench и RULER эксперименты в этой работе не воспроизведены.

Код авторов успешно работает в локальном CPU-окружении Windows. Структурные проверки подтвердили сохранение причинной маски и равномерное внимание NoPE на последовательности одинаковых токенов. Выполнено сравнение трех режимов SmolLM: исходного RoPE, удаления RoPE без обучения и рекалибровки на 4096 предъявленных токенах WikiText-2.

2. Идея статьи

RoPE поворачивает векторы запросов и ключей в attention в зависимости от позиции. Это помогает модели выучить позиционные закономерности. При увеличении длины контекста возникают невиданные во время обучения относительные фазы. Масштабирование RoPE может сохранить перплексию, одновременно ухудшив доступ к удаленной информации.

DroPE использует RoPE при предобучении, затем отключает позиционные повороты во всех слоях и продолжает языковое обучение на исходной длине последовательности. В коде авторов отключение задается cos = 1 и sin = 0. Causal mask сохраняется. Термин zero-shot context extension относится к новым длинам после рекалибровки; он не означает отсутствие дополнительного обучения вообще.

Результат работы с короткой рекалибровкой зависит от бюджета. Предъявленные здесь 4096 токенов несопоставимы с миллиардами токенов в статье. Поэтому локальная абляция проверяет исполнимость и поведение механизма, а не точность опубликованных benchmark scores.

3. Материалы авторов и опубликованные результаты

Переиспользован код custom_models/attention.py из официального SakanaAI/DroPE. Файлы сохранены без изменений в vendor/, вместе с конфигурациями обучения и исходным README. Git commit: 260a8bb9f4fda0673266ddfee7668915aade8b33. Авторство принадлежит исходному проекту.

Опубликованный checkpoint: SakanaAI/Llama-2-7b-hf-DroPE, revision 02a73325b6ed83e2fb951d68b41e50b47a258785. Скачаны конфигурация, индекс весов и код. Сервер вернул HTTP 206 и первые 1 048 576 байт первого файла весов. Общий объем весов по индексу: 13 476 831 232 байт. Попытка запуска GPU-скрипта в установленной CPU-среде остановилась на проверке CUDA; ошибка сохранена в authors_launch_attempt.log. Полный инференс не выполнен.

В таблице ниже приведены результаты авторов из таблицы 2 статьи для SmolLM-360M. Avg объединяет пять разных task scores, его нельзя читать как единую accuracy. NIAH характеризует извлечение фактов. Эти значения не измерялись локально.

Метод	Avg. score	NIAH
SmolLM	2,98	0,00
PI	6,66	0,00
RoPE-NTK	17,11	29,84
YaRN	19,94	48,25
DroPE	30,52	74,92

Опубликованный средний score DroPE примерно в 10,24 раза выше исходной SmolLM. Однако преимущество не одинаково на всех задачах: на GovReport RoPE-NTK получает 23,71, а DroPE 21,87. Для Llama-2-7B авторы сообщают средний score 26,08 у DroPE против 20,03 у base model (таблица 3).

4. Локальное окружение

Windows, Python 3.12.14, PyTorch 2.8.0+cpu, Transformers 4.56.2. PyArrow 21.0.0. Вычисления на CPU, 8 потоков, float32, seed 42. На компьютере доступны 32 ГБ RAM и NVIDIA RTX 5060 с 8 ГБ VRAM. CUDA-вариант PyTorch не использовался для измерений.

Оригинальный training launcher использует Bash, Accelerate и DeepSpeed. Вместо полного distributed trainer применен небольшой локальный цикл PyTorch и исходный класс NoPELlamaAttention. Это изменение запуска и бюджета, а не новая реализация attention. Отдельный CUDA/NF4 сценарий для авторского checkpoint приложен, но не проверен полным инференсом.

5. Протокол воспроизведения

Сначала выполнены структурные проверки на маленьком Llama Transformer: два слоя, hidden size 32, четыре головы, vocabulary 64. Исходная RoPE и NoPE модели имеют одинаковые веса. Для последовательности из 16 одинаковых токенов измерены отклонение attention от 1/i, различие скрытых состояний и градиент Q. Дополнительно изменены будущие токены и проверено, что logits префикса не меняются.

Затем загружена HuggingFaceTB/SmolLM-360M, revision 59f7ef243ee09a72cbc14cb054393a3e3b771d41. Данные: Salesforce/wikitext, wikitext-2-raw-v1, revision b08601e04326c79dfdd32d625aee71d232d685c3. Train split используется только для обучения, test split только для оценки.

Порядок состояний: (1) pretrained RoPE; (2) та же модель с заменой attention на NoPE, без обновления весов; (3) эта NoPE модель после 16 шагов AdamW, batch size 1, длина 256, LR 3e-5, weight decay 0,1, clip norm 1,0. Всего предъявлено 4096 обучающих токенов. QKNorm и scheduler не использовались.

Метрика: PPL = exp(NLL). NLL вычисляет стандартный causal LM loss по следующему токену. На каждой длине берется один test prefix из 256, 2048 или 4096 токенов. Для трех режимов внутри одной длины текст идентичен. Число target tokens равно длине минус один. Длина 4096 вдвое превышает исходный контекст SmolLM (2048). Между длинами целевые токены различаются, поэтому главный корректный контраст - между режимами на одной длине.

6. Структурные результаты

Проверка	Максимальная ошибка
NoPE attention против 1/i	0.00e+00
Различие hidden states NoPE	4.77e-07
Изменение logits префикса	0.00e+00
Абсолютный градиент Q	1.39e-10
RoPE attention против 1/i	4.32e-03

Все заданные проверки прошли. Численные остатки NoPE близки к погрешности float32. У RoPE attention на тех же одинаковых токенах заметно отличается от равномерного. Это численная проверка частного случая утверждения 3.2, а не доказательство общей теоремы о сходимости.

7. Перплексия SmolLM и интерпретация

Состояние	256	2048	4096
Исходная RoPE	22,19	9,71	63,16
Удаление без обучения	270,09	491,66	541,53
После 16 шагов	134,97	258,59	296,24

При 2048 токенах PPL исходной модели равна 9.71; сразу после отключения RoPE она увеличивается до 491.66. После микрорекалибровки PPL составляет 258.59. Удаление RoPE вызывает сильный сдвиг распределения модели. Этот запуск показывает необходимость адаптации весов.

На 4096 токенах исходная модель получает PPL 63.16, а вариант после 16 шагов 296.24. Все режимы технически обработали последовательность длиной 4096. Однако успешный прямой проход не доказывает эффективный поиск информации в полном контексте. На данном микробюджете воспроизведение улучшения длинного контекста из статьи не подтверждено.

Рекалибровка заняла 31.2 с. Первая и последняя обучающие потери: 6.466 и 5.343. Они относятся к разным обучающим примерам и не оценивают качество на test. Отдельно прошла проверка генерации 24 новых токенов.

Генерация после микрорекалибровки: The capital of France is Paris, the capital of the department of the region of the Paris, the Paris, the region of the Paris, the

8. Ограничения и вывод

Полный результат статьи не воспроизведен: отсутствуют рекалибровка на миллиардах токенов, полные LongBench/RULER выборки, сравнение PI/NTK/YaRN на локальной модели и многократные запуски. Каждая точка PPL основана на одной test-последовательности. Использован WikiText-2 вместо исходного обучающего корпуса. Числа PPL здесь не сопоставимы с таблицами статьи и не имеют доверительных интервалов.

Вывод: исходный код DroPE запускается локально и реализует заявленный механизм отключения RoPE. Проверен теоретический частный случай и выполнена воспроизводимая абляция pretrained модели. Измерения показывают, что удаления RoPE без достаточной рекалибровки недостаточно. Для проверки основного эффекта статьи нужен опубликованный обученный checkpoint и полноценная оценка длинного контекста.

9. Как повторить запуск

Откройте PowerShell в папке code. Установите Python 3.12 и выполните команды. Полные инструкции, CUDA/NF4 сценарий и описание различий находятся в README.md.

```
./setup_cpu.ps1

& ./.venv/Scripts/python.exe download_assets.py --cache cache

& ./.venv/Scripts/python.exe reproduce.py --cache cache --out results/local.json --steps 16 --seq-len 256 --device cpu
```

В архиве есть исходники, requirements, snapshot авторского attention, конфигурации, JSON с измерениями и логи. Веса и виртуальное окружение в архив не включены. В текущем рабочем пространстве скачанная SmolLM находится в work/cache/smol, среда в work/venv. Повторный запуск может дать небольшие численные расхождения при другой версии CPU/PyTorch.

10. Источники и происхождение данных

Статья, версия v1, таблицы 1–3 и приложение C
https://arxiv.org/abs/2512.12167v1

Официальный код Sakana AI, фиксированный commit
https://github.com/SakanaAI/DroPE/tree/260a8bb9f4fda0673266ddfee7668915aade8b33

Опубликованный Llama-2-7B-DroPE
https://huggingface.co/SakanaAI/Llama-2-7b-hf-DroPE

Базовый SmolLM-360M
https://huggingface.co/HuggingFaceTB/SmolLM-360M

WikiText-2
https://huggingface.co/datasets/Salesforce/wikitext

Объяснение и иллюстрации авторов
https://pub.sakana.ai/DroPE/

Авторы всех опубликованных результатов: Gelberg et al. Собственные измерения: results/local.json и results/run.log. Результаты авторов отдельно: results/authors_results.json. Проверка доступа к весам: results/authors_probe.json. Версии среды: results/environment.lock.txt. Контрольные суммы ресурсов: results/assets_sha256.json.

Авторские рисунки в презентации воспроизведены с указанием источников в заметках. Статья опубликована на arXiv с CC BY 4.0. На момент сохранения snapshot исходный GitHub repository не содержит отдельного LICENSE; авторство и исходный README сохранены, новая лицензия авторскому коду не назначается.